EcoPulse – Intelligent Municipal Waste Management System

Database Normalization Report

Document Title: Comprehensive Database Design and Normalization Technical Audit

Author / Candidate: Athulya R Mohan

Degree / Course: Master of Computer Applications (MCA) Final Year Project

Database Management System: PostgreSQL 15+ / Django ORM 5.x

Audit Scope: Active Application Schema (`accounts` , `waste` , `notifications` , `auth_user`)

Status: Verified Production-Grade Relational Schema

Executive Summary

This technical report provides a rigorous academic and architectural evaluation of the relational database schema developed for **EcoPulse**, an AI-assisted municipal solid waste tracking, grievance management, and circular-economy recycling platform.

The analysis inspects all **15 application-specific relational database tables**, documenting attribute types, candidate keys, primary and foreign key constraints, complete functional dependency sets (FD), and formal stepwise proofs across First Normal Form (1NF), Second Normal Form (2NF), and Third Normal Form (3NF). The report also justifies the deliberate engineering trade-offs (such as precomputed summary aggregates and native binary JSONB storage for YOLO computer-vision inference) implemented to balance theoretical normalization with real-world municipal operational performance.

---

1. Inventory of Active Database Tables

The EcoPulse application ecosystem operates on 15 core database tables structured across four major operational modules:

#	Table Name	Django Model	Subsystem Module	Primary Responsibility
1	`auth_user`	`User`	Django Core / Auth	Master authentication identities, credential hashes, and system permissions.
2	`accounts_ward`	`Ward`	Territory & Governance	Municipal administrative wards and local Material Collection Facility (MCF) geolocation.
3	`accounts_userprofile`	`UserProfile`	Citizen Management	Citizen profiles, contact details, ward locality mapping, and RBAC roles.
4	`accounts_staff`	`Staff`	Sanitation Operations	Sanitation personnel records, municipal designations, employment dates, and contact data.
5	`accounts_recycler`	`Recycler`	Industrial Recycling	Authorized recycling vendors, corporate details, and waste processing specializations.
6	`waste_wastepickuprequest`	`WastePickupRequest`	Waste Logistics	Citizen doorstep pickup requests, GPS coordinates, scheduled dates, and assignment state.
7	`waste_wastepickupimage`	`WastePickupImage`	AI Classification	Image evidence with YOLO deep-learning waste classification and detection matrices.
8	`waste_wastecollection`	`WasteCollection`	Field Execution	Physical proof-of-work collection records, weighed waste mass, and completion imagery.
9	`waste_wastecomplaint`	`WasteComplaint`	Grievance Redressal	Citizen complaints regarding illegal dumping or missed pickups, with priority triage.
10	`waste_staffmcfcollection`	`StaffMCFCollection`	Facility Inflow	Daily aggregate waste deposited at MCF categorized across 7 distinct waste streams.
11	`waste_mcfrecyclerassignment`	`MCFRecyclerAssignment`	Recycling Outflow	Consignments dispatched from municipal MCF hubs to certified recycling factories.

#	Table Name	Django Model	Subsystem Module	Primary Responsibility
12	`waste_payment`	`Payment`	Revenue & Billing	Citizen payments for waste pickups processed securely via Razorpay payment gateway.
13	`waste_rewardsubmission`	`RewardSubmission`	Community Incentive	Citizen green points earned through verified bottle and recyclable deposit returns.
14	`notifications_notification`	`Notification`	Event Alerting	In-app user notifications linked to pickup scheduling, payments, and complaint status.
15	`notifications_pushsubscription`	`PushSubscription`	Web Push Service	W3C Web Push subscription endpoints and ECDH encryption keys for browser push alerts.

---

2. Table Specifications: Columns, Data Types, Primary & Foreign Keys

Table 1: `auth_user`

Central authentication credentials managed by Django core auth subsystem.

- **Primary Key:** `id` (`INTEGER` , Auto-increment)

- **Alternate / Candidate Key:** `username` (`VARCHAR(150)` , Unique)

Column Name	PostgreSQL Data Type	Nullable	Constraints & Defaults	Description
`id`	`INTEGER`	No	`PRIMARY KEY`, `AUTO_INCREMENT`	Unique system user ID
`password`	`VARCHAR(128)`	No	—	PBKDF2 hashed password string
`last_login`	`TIMESTAMPTZ`	Yes	—	Timestamp of most recent user login
`is_superuser`	`BOOLEAN`	No	`DEFAULT FALSE`	Master superuser administrative flag
`username`	`VARCHAR(150)`	No	`UNIQUE`	Unique alphanumeric login username
`first_name`	`VARCHAR(150)`	No	—	User first name
`last_name`	`VARCHAR(150)`	No	—	User surname / family name
`email`	`VARCHAR(254)`	No	—	User contact and notification email
`is_staff`	`BOOLEAN`	No	`DEFAULT FALSE`	Access flag for Django administration portal
`is_active`	`BOOLEAN`	No	`DEFAULT TRUE`	Account active / inactive status
`date_joined`	`TIMESTAMPTZ`	No	—	Account creation timestamp

Table 2: `accounts_ward`

Municipal territory and Material Collection Facility (MCF) geo-boundary definition.

- **Primary Key:** `ward_id` (`INTEGER`, Auto-increment)

- **Alternate / Candidate Key:** `ward_number` (`INTEGER`, Unique)

Column Name	PostgreSQL Data Type	Nullable	Constraints & Defaults	Description
`ward_id`	`INTEGER`	No	`PRIMARY KEY`, `AUTO_INCREMENT`	Surrogate primary key
`ward_number`	`INTEGER`	Yes	`UNIQUE`	Official municipal administrative ward number
`ward_name`	`VARCHAR(100)`	No	—	Ward locality name (e.g., Central Ward 4)
`location`	`VARCHAR(255)`	No	—	Boundary territory description
`mcf_location_name`	`VARCHAR(255)`	No	—	Name of designated municipal MCF center
`mcf_latitude`	`NUMERIC(9,6)`	Yes	—	GPS latitude of municipal MCF center
`mcf_longitude`	`NUMERIC(9,6)`	Yes	—	GPS longitude of municipal MCF center

Table 3: `accounts_userprofile`

Citizen identity extension, role-based access control, and territorial alignment.

- **Primary Key:** `id` (`BIGINT` , Auto-increment)

- **Alternate / Candidate Key:** `user_id` (`INTEGER` , Unique)

- **Foreign Keys:**

- `user_id` $\rightarrow$ `auth_user(id)` (`ON DELETE CASCADE`)

- `ward_id` $\rightarrow$ `accounts_ward(ward_id)` (`ON DELETE SET NULL`)

Column Name	PostgreSQL Data Type	Nullable	Constraints & Defaults	Description
`id`	`BIGINT`	No	`PRIMARY KEY`, `AUTO_INCREMENT`	Surrogate primary key
`user_id`	`INTEGER`	No	`UNIQUE`, `FK -> auth_user`	1-to-1 link to user authentication account
`name`	`VARCHAR(100)`	No	—	Citizen full legal / display name
`phone`	`VARCHAR(15)`	Yes	—	Mobile contact number
`address`	`TEXT`	Yes	—	Residential street address
`ward_id`	`INTEGER`	Yes	`FK -> accounts_ward`	Residence municipal ward linkage
`role`	`VARCHAR(20)`	No	—	Role: `citizen`, `staff`, `admin`, `recycler`

Table 4: `accounts_staff`

Sanitation operations personnel profile and employment records.

- **Primary Key:** `staff_id` (`INTEGER`, Auto-increment)

- **Alternate / Candidate Key:** `user_id` (`INTEGER`, Unique)

- **Foreign Keys:**

- `user_id` $\rightarrow$ `auth_user(id)` (`ON DELETE CASCADE`)

Column Name	PostgreSQL Data Type	Nullable	Constraints & Defaults	Description
`staff_id`	`INTEGER`	No	`PRIMARY KEY`, `AUTO_INCREMENT`	Surrogate primary key
`user_id`	`INTEGER`	No	`UNIQUE`, `FK -> auth_user`	1-to-1 link to user account
`phone`	`VARCHAR(15)`	Yes	—	Official mobile contact number
`address`	`TEXT`	Yes	—	Residential contact address
`profile_photo`	`VARCHAR(100)`	Yes	—	Path to official ID photo upload
`designation`	`VARCHAR(50)`	No	—	Role: `Driver`, `Sanitation Worker`, `Supervisor`
`date_of_joining`	`DATE`	No	—	Municipal employment start date

Table 5: `accounts_recycler`

Authorized industrial recycling vendor credential and material specialization.

- **Primary Key:** `recycler_id` (`INTEGER`, Auto-increment)

- **Alternate / Candidate Key:** `user_id` (`INTEGER`, Unique)

- **Foreign Keys:**

- `user_id` $\rightarrow$ `auth_user(id)` (`ON DELETE CASCADE`)

Column Name	PostgreSQL Data Type	Nullable	Constraints & Defaults	Description
`recycler_id`	`INTEGER`	No	`PRIMARY KEY`, `AUTO_INCREMENT`	Surrogate primary key
`user_id`	`INTEGER`	No	`UNIQUE`, `FK -> auth_user`	1-to-1 link to user account
`organization`	`VARCHAR(100)`	No	—	Registered corporate business name
`waste_type`	`VARCHAR(50)`	No	—	Material specialization (`Plastic`, `Metal`, `E-Waste`)
`address`	`TEXT`	Yes	—	Processing plant / industrial address
`profile_photo`	`VARCHAR(100)`	Yes	—	Corporate logo / factory photo upload
`date_of_registration`	`DATE`	No	—	Municipal board licensing registration date

Table 6: `waste_wastepickuprequest`

Citizen doorstep pickup booking lifecycle and routing dispatch.

- **Primary Key:** `id` (`BIGINT`, Auto-increment)

- **Foreign Keys:**

- `user_id` $\rightarrow$ `auth_user(id)` (`ON DELETE CASCADE`)

- `assigned_staff_id` $\rightarrow$ `accounts_staff(staff_id)` (`ON DELETE SET NULL`)

Column Name	PostgreSQL Data Type	Nullable	Constraints & Defaults	Description
`id`	`BIGINT`	No	`PRIMARY KEY`, `AUTO_INCREMENT`	Unique pickup request ID
`user_id`	`INTEGER`	No	`FK -> auth_user`	Citizen who submitted the request
`location_name`	`VARCHAR(255)`	No	—	Landmark / street address description
`latitude`	`NUMERIC(9,6)`	No	—	GPS latitude for route optimization
`longitude`	`NUMERIC(9,6)`	No	—	GPS longitude for route optimization
`status`	`VARCHAR(20)`	No	`DEFAULT 'requested'`	State: `requested`, `assigned`, `completed`, `pending`
`assigned_staff_id`	`INTEGER`	Yes	`FK -> accounts_staff`	Assigned sanitation staff driver
`staff_marked_pending`	`BOOLEAN`	No	`DEFAULT FALSE`	Inability-to-collect flag toggled by driver
`pending_reason`	`TEXT`	Yes	—	Driver explanation for delayed collection
`scheduled_pickup_date`	`DATE`	Yes	—	Scheduled date for collection truck route
`created_at`	`TIMESTAMPTZ`	No	—	Booking submission timestamp
`updated_at`	`TIMESTAMPTZ`	No	—	Status modification timestamp

Table 7: `waste_wastepickupimage`

Uploaded waste photography and AI YOLO deep-learning classification results.

- **Primary Key:** `id` (`BIGINT`, Auto-increment)

- **Foreign Keys:**

- `pickup_request_id` $\rightarrow$ `waste_wastepickuprequest(id)` (`ON DELETE CASCADE`)

Column Name	PostgreSQL Data Type	Nullable	Constraints & Defaults	Description
`id`	`BIGINT`	No	`PRIMARY KEY`, `AUTO_INCREMENT`	Unique image record ID
`pickup_request_id`	`BIGINT`	No	`FK -> waste_wastepickuprequest`	Associated pickup request
`image`	`VARCHAR(100)`	No	—	File path to waste photo
`waste_type`	`VARCHAR(50)`	Yes	—	YOLO primary predicted waste category
`confidence`	`DOUBLE PRECISION`	Yes	—	Model classification confidence (0.00 – 1.00)
`detections`	`JSONB`	No	`DEFAULT '[]'`	Bounding box coordinates and class predictions
`created_at`	`TIMESTAMPTZ`	No	—	Analysis creation timestamp

Table 8: `waste_wastecollection`

Physical collection completion execution and weighed solid waste mass.

- **Primary Key:** `id` (`BIGINT` , Auto-increment)

- **Alternate / Candidate Key:** `pickup_request_id` (`BIGINT` , Unique)

- **Foreign Keys:**

- `pickup_request_id` $\rightarrow$ `waste_wastepickuprequest(id)` (`ON DELETE CASCADE`)

- `staff_id` $\rightarrow$ `accounts_staff(staff_id)` (`ON DELETE CASCADE`)

Column Name	PostgreSQL Data Type	Nullable	Constraints & Defaults	Description
`id`	`BIGINT`	No	`PRIMARY KEY`, `AUTO_INCREMENT`	Unique collection fulfillment ID
`pickup_request_id`	`BIGINT`	No	`UNIQUE`, `FK`	1-to-1 fulfillment of pickup request
`staff_id`	`INTEGER`	No	`FK -> accounts_staff`	Crew member who weighed and collected waste
`completion_image`	`VARCHAR(100)`	No	—	Proof-of-work photograph at collection site
`estimated_weight`	`NUMERIC(7,2)`	No	—	Weighed mass in kilograms
`completed_at`	`TIMESTAMPTZ`	No	—	Collection timestamp

Table 9: `waste_wastecomplaint`

Public grievance redressal, hazard reports, and resolution proof.

- **Primary Key:** `id` (`BIGINT`, Auto-increment)

- **Foreign Keys:**

- `user_id` $\rightarrow$ `auth_user(id)` (`ON DELETE CASCADE`)

- `assigned_staff_id` $\rightarrow$ `accounts_staff(staff_id)` (`ON DELETE SET NULL`)

Column Name	PostgreSQL Data Type	Nullable	Constraints & Defaults	Description
`id`	`BIGINT`	No	`PRIMARY KEY`, `AUTO_INCREMENT`	Unique complaint tracking ID
`user_id`	`INTEGER`	No	`FK -> auth_user`	Citizen who lodged the grievance
`image`	`VARCHAR(100)`	Yes	—	Evidence photo of illegal dumping
`location_name`	`VARCHAR(255)`	No	—	Street name / landmark of incident
`latitude`	`NUMERIC(9,6)`	Yes	—	GPS incident latitude
`longitude`	`NUMERIC(9,6)`	Yes	—	GPS incident longitude
`description`	`TEXT`	No	—	Narrative description of issue
`priority`	`VARCHAR(20)`	No	`DEFAULT 'medium'`	Priority level: `low`, `medium`, `high`
`status`	`VARCHAR(20)`	No	`DEFAULT 'pending'`	State: `pending`, `assigned`, `resolved`
`assigned_staff_id`	`INTEGER`	Yes	`FK -> accounts_staff`	Assigned cleanup staff worker
`proof_image`	`VARCHAR(100)`	Yes	—	Post-cleanup verification photo
`resolved_at`	`TIMESTAMPTZ`	Yes	—	Resolution completion timestamp
`created_at`	`TIMESTAMPTZ`	No	—	Complaint lodgement timestamp
`updated_at`	`TIMESTAMPTZ`	No	—	Last status update timestamp

Table 10: `waste_staffmcfcollection`

Facility daily intake log categorized into 7 segregated waste streams.

- **Primary Key:** `id` (`BIGINT`, Auto-increment)

- **Foreign Keys:**

- `staff_id` $\rightarrow$ `accounts_staff(staff_id)` (`ON DELETE CASCADE`)

Column Name	PostgreSQL Data Type	Nullable	Constraints & Defaults	Description
`id`	`BIGINT`	No	`PRIMARY KEY`, `AUTO_INCREMENT`	Unique daily MCF intake ID
`staff_id`	`INTEGER`	No	`FK -> accounts_staff`	Staff member depositing waste load
`date`	`DATE`	No	—	Intake date
`paper`	`NUMERIC(7,2)`	No	`DEFAULT 0.00`	Paper waste mass (kg)
`plastic`	`NUMERIC(7,2)`	No	`DEFAULT 0.00`	Plastic waste mass (kg)
`glass`	`NUMERIC(7,2)`	No	`DEFAULT 0.00`	Glass waste mass (kg)
`metal`	`NUMERIC(7,2)`	No	`DEFAULT 0.00`	Metal waste mass (kg)
`organic`	`NUMERIC(7,2)`	No	`DEFAULT 0.00`	Wet/organic compostable waste mass (kg)
`electronics`	`NUMERIC(7,2)`	No	`DEFAULT 0.00`	Electronic e-waste mass (kg)
`miscellaneous`	`NUMERIC(7,2)`	No	`DEFAULT 0.00`	Unsorted inert waste mass (kg)
`total_quantity`	`NUMERIC(8,2)`	No	`DEFAULT 0.00`	Controlled cached sum of 7 streams (kg)
`created_at`	`TIMESTAMPTZ`	No	—	Logging timestamp
`updated_at`	`TIMESTAMPTZ`	No	—	Modification timestamp

Table 11: `waste_mcfrecyclerassignment`

Material dispatch manifest from municipal MCF facility to commercial recyclers.

- **Primary Key:** `id` (`BIGINT`, Auto-increment)

- **Foreign Keys:**

- `recycler_id` $\rightarrow$ `accounts_recycler(recycler_id)` (`ON DELETE CASCADE`)

- `assigned_by_id` $\rightarrow$ `auth_user(id)` (`ON DELETE SET NULL`)

Column Name	PostgreSQL Data Type	Nullable	Constraints & Defaults	Description
`id`	`BIGINT`	No	`PRIMARY KEY`, `AUTO_INCREMENT`	Unique consignment ID
`date`	`DATE`	No	—	Dispatch date
`waste_type`	`VARCHAR(50)`	No	—	Waste stream type allocated
`total_quantity`	`NUMERIC(8,2)`	No	—	Dispatched consignment tonnage / kg
`recycler_id`	`INTEGER`	No	`FK -> accounts_recycler`	Licensed destination recycler vendor
`assigned_by_id`	`INTEGER`	Yes	`FK -> auth_user`	MCF supervisor / admin who authorized shipment
`status`	`VARCHAR(20)`	No	`DEFAULT 'assigned'`	Logistics status: `assigned`, `completed`
`recyclable_quantity`	`NUMERIC(8,2)`	Yes	—	Actual processed yield reported by factory
`note`	`TEXT`	Yes	—	Special handling notes / manifest terms
`completed_at`	`TIMESTAMPTZ`	Yes	—	Factory processing completion timestamp
`recycling_image`	`VARCHAR(100)`	Yes	—	Processing certificate / weight receipt photo
`created_at`	`TIMESTAMPTZ`	No	—	Allocation creation timestamp
`updated_at`	`TIMESTAMPTZ`	No	—	Last update timestamp

Table 12: `waste_payment`

Citizen service fee invoices and Razorpay transaction settlements.

- **Primary Key:** `id` (`BIGINT`, Auto-increment)

- **Alternate / Candidate Keys:**

- `payment_id` (`VARCHAR(100)`, Unique)

- `pickup_request_id` (`BIGINT`, Unique)

- **Foreign Keys:**

- `pickup_request_id` $\rightarrow$ `waste_wastepickuprequest(id)` (`ON DELETE CASCADE`)

- `user_id` $\rightarrow$ `auth_user(id)` (`ON DELETE CASCADE`)

Column Name	PostgreSQL Data Type	Nullable	Constraints & Defaults	Description
`id`	`BIGINT`	No	`PRIMARY KEY`, `AUTO_INCREMENT`	Unique payment record ID
`pickup_request_id`	`BIGINT`	No	`UNIQUE`, `FK`	1-to-1 invoice link to pickup request
`user_id`	`INTEGER`	No	`FK -> auth_user`	Citizen who paid the service charge
`amount`	`NUMERIC(8,2)`	No	—	Billed fee amount (INR)
`payment_status`	`VARCHAR(20)`	No	`DEFAULT 'pending'`	State: `pending`, `completed`, `failed`
`payment_id`	`VARCHAR(100)`	No	`UNIQUE`	Internal system payment tracking reference
`transaction_id`	`VARCHAR(100)`	Yes	—	Razorpay bank authorization reference
`order_id`	`VARCHAR(100)`	Yes	—	Razorpay order session identifier
`payment_method`	`VARCHAR(50)`	No	`DEFAULT 'razorpay'`	Channel: `razorpay`, `upi`, `netbanking`
`payment_date`	`TIMESTAMPTZ`	Yes	—	Payment settlement timestamp
`created_at`	`TIMESTAMPTZ`	No	—	Invoice generation timestamp
`updated_at`	`TIMESTAMPTZ`	No	—	Status update timestamp

Table 13: `waste_rewardsubmission`

Reverse vending / recyclable bottle deposits and citizen incentive vouchers.

- **Primary Key:** `id` (`BIGINT`, Auto-increment)

- **Foreign Keys:**

- `user_id` $\rightarrow$ `auth_user(id)` (`ON DELETE CASCADE`)

- `reward_assigned_by_id` $\rightarrow$ `auth_user(id)` (`ON DELETE SET NULL`)

Column Name	PostgreSQL Data Type	Nullable	Constraints & Defaults	Description
`id`	`BIGINT`	No	`PRIMARY KEY`, `AUTO_INCREMENT`	Unique reward submission ID
`user_id`	`INTEGER`	No	`FK -> auth_user`	Citizen recipient of green points
`photo`	`VARCHAR(100)`	No	—	Photo evidence of bottle deposit
`date`	`DATE`	Yes	—	Deposit submission date
`bin_location`	`VARCHAR(255)`	Yes	—	Automated reverse vending kiosk location
`waste_type`	`VARCHAR(50)`	Yes	—	Recyclable class (`Plastic Bottles`, `Cans`)
`estimated_quantity`	`NUMERIC(7,2)`	Yes	—	Deposited unit count or weight
`reward_amount`	`NUMERIC(8,2)`	Yes	—	Monetary coupon equivalent (INR)
`reward_assigned_at`	`TIMESTAMPTZ`	Yes	—	Approval and point grant timestamp
`reward_assigned_by_id`	`INTEGER`	Yes	`FK -> auth_user`	Verifying municipal officer ID
`points`	`INTEGER`	Yes	—	Eco-points credited to citizen wallet
`valid_from`	`TIMESTAMPTZ`	Yes	—	Voucher validity start timestamp
`valid_till`	`TIMESTAMPTZ`	Yes	—	Voucher expiration timestamp
`claimed_at`	`TIMESTAMPTZ`	Yes	—	Citizen redemption timestamp
`status`	`VARCHAR(20)`	No	`DEFAULT 'pending'`	State: `pending`, `assigned`, `claimed`
`created_at`	`TIMESTAMPTZ`	No	—	Submission timestamp
`updated_at`	`TIMESTAMPTZ`	No	—	Last update timestamp

Table 14: `notifications_notification`

User alerting engine with deep-linking to pickups and complaints.

- **Primary Key:** `id` (`BIGINT`, Auto-increment)

- **Foreign Keys:**

- `recipient_id` $\rightarrow$ `auth_user(id)` (`ON DELETE CASCADE`)

- `related_pickup_id` $\rightarrow$ `waste_wastepickuprequest(id)` (`ON DELETE SET NULL`)

- `related_complaint_id` $\rightarrow$ `waste_wastecomplaint(id)` (`ON DELETE SET NULL`)

Column Name	PostgreSQL Data Type	Nullable	Constraints & Defaults	Description
`id`	`BIGINT`	No	`PRIMARY KEY`, `AUTO_INCREMENT`	Unique notification ID
`recipient_id`	`INTEGER`	No	`FK -> auth_user`	Target recipient user
`title`	`VARCHAR(150)`	No	—	Notification subject line
`message`	`TEXT`	No	—	Complete notification message body
`notification_type`	`VARCHAR(30)`	No	—	Type: `pickup`, `complaint`, `payment`, `system`
`related_pickup_id`	`BIGINT`	Yes	`FK -> waste_wastepickuprequest`	Optional related pickup reference
`related_complaint_id`	`BIGINT`	Yes	`FK -> waste_wastecomplaint`	Optional related complaint reference
`action_url`	`VARCHAR(255)`	No	`DEFAULT ''`	Client UI deep-link route
`is_read`	`BOOLEAN`	No	`DEFAULT FALSE`	Read status toggle
`created_at`	`TIMESTAMPTZ`	No	—	Notification generation timestamp
`updated_at`	`TIMESTAMPTZ`	No	—	Read / update timestamp

Table 15: `notifications_pushsubscription`

W3C Web Push cryptographic keys and browser delivery endpoints.

- **Primary Key:** `id` (`BIGINT`, Auto-increment)

- **Alternate / Candidate Key:** `endpoint` (`TEXT`, Unique)

- **Foreign Keys:**

- `user_id` $\rightarrow$ `auth_user(id)` (`ON DELETE CASCADE`)

Column Name	PostgreSQL Data Type	Nullable	Constraints & Defaults	Description
`id`	`BIGINT`	No	`PRIMARY KEY`, `AUTO_INCREMENT`	Unique push subscription ID
`user_id`	`INTEGER`	No	`FK -> auth_user`	Subscriber user account
`endpoint`	`TEXT`	No	`UNIQUE`	Unique browser push gateway URL
`p256dh`	`TEXT`	No	—	ECDH P-256 public key (Base64)
`auth`	`TEXT`	No	—	Shared authentication secret (Base64)
`user_agent`	`TEXT`	No	—	Browser and platform user agent string
`is_active`	`BOOLEAN`	No	`DEFAULT TRUE`	Subscription activation toggle
`created_at`	`TIMESTAMPTZ`	No	—	Device registration timestamp
`updated_at`	`TIMESTAMPTZ`	No	—	Last key refresh timestamp

3. Relationships & Referential Integrity Rules

The relational structure of EcoPulse enforces relational consistency and prevents orphan records using standard PostgreSQL `FOREIGN KEY` constraints configured through Django ORM:

Parent Table	Child Table	Foreign Key Column	Cardinality	On-Delete Action
`auth_user`	`accounts_userprofile`	`user_id`	1 : 1	`CASCADE`
`accounts_ward`	`accounts_userprofile`	`ward_id`	N : 1	`SET_NULL`
`auth_user`	`accounts_staff`	`user_id`	1 : 1	`CASCADE`
`auth_user`	`accounts_recycler`	`user_id`	1 : 1	`CASCADE`
`auth_user`	`waste_wastepickuprequest`	`user_id`	N : 1	`CASCADE`
`accounts_staff`	`waste_wastepickuprequest`	`assigned_staff_id`	N : 1	`SET_NULL`
`waste_wastepickuprequest`	`waste_wastepickupimage`	`pickup_request_id`	N : 1	`CASCADE`
`waste_wastepickuprequest`	`waste_wastecollection`	`pickup_request_id`	1 : 1	`CASCADE`

Parent Table	Child Table	Foreign Key Column	Cardinality	On-Delete Action
`accounts_staff`	`waste_wastecollection`	`staff_id`	N : 1	`CASCADE`
`auth_user`	`waste_wastecomplaint`	`user_id`	N : 1	`CASCADE`
`accounts_staff`	`waste_wastecomplaint`	`assigned_staff_id`	N : 1	`SET_NULL`
`accounts_staff`	`waste_staffmcfcollection`	`staff_id`	N : 1	`CASCADE`
`accounts_recycler`	`waste_mcfrecyclerassignment`	`recycler_id`	N : 1	`CASCADE`
`auth_user`	`waste_mcfrecyclerassignment`	`assigned_by_id`	N : 1	`SET_NULL`
`waste_wastepickuprequest`	`waste_payment`	`pickup_request_id`	1 : 1	`CASCADE`
`auth_user`	`waste_payment`	`user_id`	N : 1	`CASCADE`

Parent Table	Child Table	Foreign Key Column	Cardinality	On-Delete Action
`auth_user`	`waste_rewardsubmission`	`user_id`	N : 1	`CASCADE`
`auth_user`	`waste_rewardsubmission`	`reward_assigned_by_id`	N : 1	`SET_NULL`
`auth_user`	`notifications_notification`	`recipient_id`	N : 1	`CASCADE`
`waste_wastepickuprequest`	`notifications_notification`	`related_pickup_id`	N : 1	`SET_NULL`
`waste_wastecomplaint`	`notifications_notification`	`related_complaint_id`	N : 1	`SET_NULL`
`auth_user`	`notifications_pushsubscription`	`user_id`	N : 1	`CASCADE`

4. Formal Functional Dependencies (FDs) Table-by-Table

A functional dependency $X \rightarrow Y$ states that whenever two tuples share the same value for attribute set X, they must also share the same value for attribute set Y.

```
1. auth_user
   FD 1.1: {id} -> {username, password, last_login, is_superuser, first_name, last_name, email,
is_staff, is_active, date_joined}
   FD 1.2: {username} -> {id, password, last_login, is_superuser, first_name, last_name, email,
is_staff, is_active, date_joined}

2. accounts_ward
   FD 2.1: {ward_id} -> {ward_number, ward_name, location, mcf_location_name, mcf_latitude,
mcf_longitude}
   FD 2.2: {ward_number} -> {ward_id, ward_name, location, mcf_location_name, mcf_latitude,
mcf_longitude}

3. accounts_userprofile
   FD 3.1: {id} -> {user_id, name, phone, address, ward_id, role}
   FD 3.2: {user_id} -> {id, name, phone, address, ward_id, role}

4. accounts_staff
   FD 4.1: {staff_id} -> {user_id, phone, address, profile_photo, designation, date_of_joining}
   FD 4.2: {user_id} -> {staff_id, phone, address, profile_photo, designation, date_of_joining}

5. accounts_recycler
   FD 5.1: {recycler_id} -> {user_id, organization, waste_type, address, profile_photo,
date_of_registration}
   FD 5.2: {user_id} -> {recycler_id, organization, waste_type, address, profile_photo,
date_of_registration}

6. waste_wastepickuprequest
   FD 6.1: {id} -> {user_id, location_name, latitude, longitude, status, assigned_staff_id,
staff_marked_pending, pending_reason, scheduled_pickup_date, created_at, updated_at}

7. waste_wastepickupimage
   FD 7.1: {id} -> {pickup_request_id, image, waste_type, confidence, detections, created_at}

8. waste_wastecollection
   FD 8.1: {id} -> {pickup_request_id, staff_id, completion_image, estimated_weight, completed_at}
   FD 8.2: {pickup_request_id} -> {id, staff_id, completion_image, estimated_weight, completed_at}

9. waste_wastecomplaint
   FD 9.1: {id} -> {user_id, image, location_name, latitude, longitude, description, priority,
status, assigned_staff_id, proof_image, resolved_at, created_at, updated_at}

10. waste_staffmcfcollection
   FD 10.1: {id} -> {staff_id, date, paper, plastic, glass, metal, organic, electronics,
miscellaneous, total_quantity, created_at, updated_at}
   FD 10.2: {paper, plastic, glass, metal, organic, electronics, miscellaneous} ->
{total_quantity}  (Controlled Derived Dependency)

11. waste_mcfrecyclerassignment
   FD 11.1: {id} -> {date, waste_type, total_quantity, recycler_id, assigned_by_id, status,
recyclable_quantity, note, completed_at, recycling_image, created_at, updated_at}

12. waste_payment
   FD 12.1: {id} -> {pickup_request_id, user_id, amount, payment_status, payment_id,
```

```
transaction_id, order_id, payment_method, payment_date, created_at, updated_at}
   FD 12.2: {payment_id} -> {id, pickup_request_id, user_id, amount, payment_status,
transaction_id, order_id, payment_method, payment_date, created_at, updated_at}
   FD 12.3: {pickup_request_id} -> {id, user_id, amount, payment_status, payment_id,
transaction_id, order_id, payment_method, payment_date, created_at, updated_at}

13. waste_rewardsubmission
   FD 13.1: {id} -> {user_id, photo, date, bin_location, waste_type, estimated_quantity,
reward_amount, reward_assigned_at, reward_assigned_by_id, points, valid_from, valid_till,
claimed_at, status, created_at, updated_at}

14. notifications_notification
   FD 14.1: {id} -> {recipient_id, title, message, notification_type, related_pickup_id,
related_complaint_id, action_url, is_read, created_at, updated_at}

15. notifications_pushsubscription
   FD 15.1: {id} -> {user_id, endpoint, p256dh, auth, user_agent, is_active, created_at,
updated_at}
   FD 15.2: {endpoint} -> {id, user_id, p256dh, auth, user_agent, is_active, created_at,
updated_at}
```

---

5. First Normal Form (1NF) Analysis

1NF Criteria

A relation R satisfies 1NF if and only if:

1. Every attribute domain contains only atomic, indivisible values.

2. The relation contains no repeating groups or nested arrays of attributes.

3. Every row possesses a distinct identity established by a Primary Key.

Evaluation of EcoPulse Tables

- **Scalar Domains:** GPS locations are decomposed into discrete numerical columns (`latitude` and `longitude`), avoiding combined comma-separated string encodings.

- **Fixed-Category Deconstruction:** In `waste_staffmcfcollection` , rather than storing a non-atomic list of waste streams, 7 distinct numeric columns (`paper` , `plastic` , `glass` , `metal` , `organic` , `electronics` , `miscellaneous`) are defined.

- **Handling of Semi-Structured AI Inference Data:** In `waste_wastepickupimage` , the `detections` column stores bounding-box detection matrices. Under modern PostgreSQL (SQL:2016), `JSONB` is treated as a native binary scalar type. Because bounding box data is produced as a composite output from deep learning inference and rendered atomically on client viewports, storing it as a `JSONB` document avoids thousands of unnecessary child rows while preserving relational integrity.

All 15 tables satisfy First Normal Form.

---

6. Second Normal Form (2NF) Analysis

2NF Criteria

A relation R satisfies 2NF if and only if:

1. R is in 1NF.

2. Every non-prime attribute is **fully functionally dependent** on the entirety of every candidate key (no partial dependencies on a proper subset of a composite candidate key).

Mathematical Verification for EcoPulse

A partial dependency can only exist when a candidate key is **composite** ($|K| \ge 2$), such that $X \subset K$ determines a non-prime attribute $Y \notin K$.

Across all 15 tables in EcoPulse:

- Every table designates a **single-attribute surrogate primary key** (`id` , `ward_id` , `staff_id` , `recycler_id`).

- All secondary candidate keys (`username` , `ward_number` , `endpoint` , `payment_id` , `pickup_request_id`) consist of **exactly one attribute**.

- Since no composite candidate keys exist in the schema, the condition for partial dependency cannot arise:

$$\forall K \in \text{CandidateKeys}(R), \quad |K| = 1 \implies \nexists X \subset K \text{ such that } X \neq \emptyset \text{ and } X \rightarrow Y$$

Therefore, all 15 tables automatically and strictly satisfy Second Normal Form (2NF).

---

7. Third Normal Form (3NF) Analysis

3NF Criteria

A relation R satisfies 3NF if and only if:

1. R is in 2NF.

2. For every non-trivial functional dependency $X \rightarrow Y$ in R:

- X is a superkey of R, OR

- Y is a prime attribute of R (every attribute in Y is part of some candidate key).

Equivalently: **No non-prime attribute is transitively dependent on a candidate key.**

Analysis of Non-Key Dependencies

- **13 of 15 Tables:** Every non-prime attribute depends solely on candidate keys. No non-prime attribute determines another non-prime attribute.

- **Avoidance of Derived Fields via `@property` :** In `waste_rewardsubmission` , green points are converted to bottle counts via the formula `points // 5` . Rather than persisting a redundant `bottle_count` column in

the database (which would introduce a transitive dependency $\{id\} \rightarrow \{points\} \rightarrow \{bottle_count\}$), EcoPulse computes this via a dynamic Python model `@property`.

- **Controlled Variance in `waste_staffmcfcollection`:** In `waste_staffmcfcollection`, the sum of 7 category quantities determines `total_quantity`:

$$\{paper, plastic, glass, metal, organic, electronics, miscellaneous\} \rightarrow \{total_quantity\}$$

This controlled derived dependency is intentionally preserved to optimize analytical dashboard queries, with consistency enforced via Django's `save()` method.

---

8. Final Normalized Relational Schema

```
 -- 1. Base User Credentials
auth_user (
    id INT PRIMARY KEY,
    username VARCHAR(150) UNIQUE NOT NULL,
    password VARCHAR(128) NOT NULL,
    first_name VARCHAR(150),
    last_name VARCHAR(150),
    email VARCHAR(254),
    is_superuser BOOLEAN NOT NULL,
    is_staff BOOLEAN NOT NULL,
    is_active BOOLEAN NOT NULL,
    date_joined TIMESTAMPTZ NOT NULL,
    last_login TIMESTAMPTZ
);

-- 2. Municipal Wards
accounts_ward (
    ward_id INT PRIMARY KEY,
    ward_number INT UNIQUE,
    ward_name VARCHAR(100) NOT NULL,
    location VARCHAR(255) NOT NULL,
    mcf_location_name VARCHAR(255) NOT NULL,
    mcf_latitude NUMERIC(9,6),
    mcf_longitude NUMERIC(9,6)
);

-- 3. Citizen Profiles
accounts_userprofile (
    id BIGINT PRIMARY KEY,
    user_id INT UNIQUE NOT NULL REFERENCES auth_user(id) ON DELETE CASCADE,
    ward_id INT REFERENCES accounts_ward(ward_id) ON DELETE SET NULL,
    name VARCHAR(100) NOT NULL,
    phone VARCHAR(15),
    address TEXT,
    role VARCHAR(20) NOT NULL
);

-- 4. Sanitation Staff
accounts_staff (
    staff_id INT PRIMARY KEY,
    user_id INT UNIQUE NOT NULL REFERENCES auth_user(id) ON DELETE CASCADE,
    phone VARCHAR(15),
    address TEXT,
    profile_photo VARCHAR(100),
    designation VARCHAR(50) NOT NULL,
    date_of_joining DATE NOT NULL
);

-- 5. Industrial Recyclers
accounts_recycler (
```

```
    recycler_id INT PRIMARY KEY,
    user_id INT UNIQUE NOT NULL REFERENCES auth_user(id) ON DELETE CASCADE,
    organization VARCHAR(100) NOT NULL,
    waste_type VARCHAR(50) NOT NULL,
    address TEXT,
    profile_photo VARCHAR(100),
    date_of_registration DATE NOT NULL
);

-- 6. Waste Pickup Requests
waste_wastepickuprequest (
    id BIGINT PRIMARY KEY,
    user_id INT NOT NULL REFERENCES auth_user(id) ON DELETE CASCADE,
    assigned_staff_id INT REFERENCES accounts_staff(staff_id) ON DELETE SET NULL,
    location_name VARCHAR(255) NOT NULL,
    latitude NUMERIC(9,6) NOT NULL,
    longitude NUMERIC(9,6) NOT NULL,
    status VARCHAR(20) NOT NULL,
    staff_marked_pending BOOLEAN NOT NULL,
    pending_reason TEXT,
    scheduled_pickup_date DATE,
    created_at TIMESTAMPTZ NOT NULL,
    updated_at TIMESTAMPTZ NOT NULL
);

-- 7. AI Computer Vision Detections
waste_wastepickupimage (
    id BIGINT PRIMARY KEY,
    pickup_request_id BIGINT NOT NULL REFERENCES waste_wastepickuprequest(id) ON DELETE CASCADE,
    image VARCHAR(100) NOT NULL,
    waste_type VARCHAR(50),
    confidence DOUBLE PRECISION,
    detections JSONB NOT NULL,
    created_at TIMESTAMPTZ NOT NULL
);

-- 8. Pickup Completion Proofs
waste_wastecollection (
    id BIGINT PRIMARY KEY,
    pickup_request_id BIGINT UNIQUE NOT NULL REFERENCES waste_wastepickuprequest(id) ON DELETE
CASCADE,
    staff_id INT NOT NULL REFERENCES accounts_staff(staff_id) ON DELETE CASCADE,
    completion_image VARCHAR(100) NOT NULL,
    estimated_weight NUMERIC(7,2) NOT NULL,
    completed_at TIMESTAMPTZ NOT NULL
);

-- 9. Citizen Complaints
waste_wastecomplaint (
    id BIGINT PRIMARY KEY,
    user_id INT NOT NULL REFERENCES auth_user(id) ON DELETE CASCADE,
    assigned_staff_id INT REFERENCES accounts_staff(staff_id) ON DELETE SET NULL,
    image VARCHAR(100),
    location_name VARCHAR(255) NOT NULL,
```

```
    latitude NUMERIC(9,6),
    longitude NUMERIC(9,6),
    description TEXT NOT NULL,
    priority VARCHAR(20) NOT NULL,
    status VARCHAR(20) NOT NULL,
    proof_image VARCHAR(100),
    resolved_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ NOT NULL,
    updated_at TIMESTAMPTZ NOT NULL
);

-- 10. Daily MCF Waste Intake
waste_staffmcfcollection (
    id BIGINT PRIMARY KEY,
    staff_id INT NOT NULL REFERENCES accounts_staff(staff_id) ON DELETE CASCADE,
    date DATE NOT NULL,
    paper NUMERIC(7,2) NOT NULL,
    plastic NUMERIC(7,2) NOT NULL,
    glass NUMERIC(7,2) NOT NULL,
    metal NUMERIC(7,2) NOT NULL,
    organic NUMERIC(7,2) NOT NULL,
    electronics NUMERIC(7,2) NOT NULL,
    miscellaneous NUMERIC(7,2) NOT NULL,
    total_quantity NUMERIC(8,2) NOT NULL,
    created_at TIMESTAMPTZ NOT NULL,
    updated_at TIMESTAMPTZ NOT NULL
);

-- 11. Recycler Consignment Dispatches
waste_mcfrecyclerassignment (
    id BIGINT PRIMARY KEY,
    recycler_id INT NOT NULL REFERENCES accounts_recycler(recycler_id) ON DELETE CASCADE,
    assigned_by_id INT REFERENCES auth_user(id) ON DELETE SET NULL,
    date DATE NOT NULL,
    waste_type VARCHAR(50) NOT NULL,
    total_quantity NUMERIC(8,2) NOT NULL,
    status VARCHAR(20) NOT NULL,
    recyclable_quantity NUMERIC(8,2),
    note TEXT,
    completed_at TIMESTAMPTZ,
    recycling_image VARCHAR(100),
    created_at TIMESTAMPTZ NOT NULL,
    updated_at TIMESTAMPTZ NOT NULL
);

-- 12. Payment Invoices
waste_payment (
    id BIGINT PRIMARY KEY,
    pickup_request_id BIGINT UNIQUE NOT NULL REFERENCES waste_wastepickuprequest(id) ON DELETE
CASCADE,
    user_id INT NOT NULL REFERENCES auth_user(id) ON DELETE CASCADE,
    payment_id VARCHAR(100) UNIQUE NOT NULL,
    amount NUMERIC(8,2) NOT NULL,
    payment_status VARCHAR(20) NOT NULL,
```

```
    transaction_id VARCHAR(100),
    order_id VARCHAR(100),
    payment_method VARCHAR(50) NOT NULL,
    payment_date TIMESTAMPTZ,
    created_at TIMESTAMPTZ NOT NULL,
    updated_at TIMESTAMPTZ NOT NULL
);

-- 13. Reward Submissions
waste_rewardsubmission (
    id BIGINT PRIMARY KEY,
    user_id INT NOT NULL REFERENCES auth_user(id) ON DELETE CASCADE,
    reward_assigned_by_id INT REFERENCES auth_user(id) ON DELETE SET NULL,
    photo VARCHAR(100) NOT NULL,
    date DATE,
    bin_location VARCHAR(255),
    waste_type VARCHAR(50),
    estimated_quantity NUMERIC(7,2),
    reward_amount NUMERIC(8,2),
    reward_assigned_at TIMESTAMPTZ,
    points INT,
    valid_from TIMESTAMPTZ,
    valid_till TIMESTAMPTZ,
    claimed_at TIMESTAMPTZ,
    status VARCHAR(20) NOT NULL,
    created_at TIMESTAMPTZ NOT NULL,
    updated_at TIMESTAMPTZ NOT NULL
);

-- 14. Notifications
notifications_notification (
    id BIGINT PRIMARY KEY,
    recipient_id INT NOT NULL REFERENCES auth_user(id) ON DELETE CASCADE,
    related_pickup_id BIGINT REFERENCES waste_wastepickuprequest(id) ON DELETE SET NULL,
    related_complaint_id BIGINT REFERENCES waste_wastecomplaint(id) ON DELETE SET NULL,
    title VARCHAR(150) NOT NULL,
    message TEXT NOT NULL,
    notification_type VARCHAR(30) NOT NULL,
    action_url VARCHAR(255) NOT NULL,
    is_read BOOLEAN NOT NULL,
    created_at TIMESTAMPTZ NOT NULL,
    updated_at TIMESTAMPTZ NOT NULL
);

-- 15. Web Push Subscriptions
notifications_pushsubscription (
    id BIGINT PRIMARY KEY,
    user_id INT NOT NULL REFERENCES auth_user(id) ON DELETE CASCADE,
    endpoint TEXT UNIQUE NOT NULL,
    p256dh TEXT NOT NULL,
    auth TEXT NOT NULL,
    user_agent TEXT NOT NULL,
    is_active BOOLEAN NOT NULL,
    created_at TIMESTAMPTZ NOT NULL,
```

```
    updated_at TIMESTAMPTZ NOT NULL
);
```

9. Controlled Denormalization & Engineering Trade-offs

Implementation Area	Theoretical Normal Form	Actual Implementation	Performance & Architectural Rationale
`waste_staffmcfcollection.total_quantity`	Pure 3NF requires dynamic SQL summation of 7 sub-columns.	Stored sum updated on model `save()`.	Speeds up municipal dashboard queries by avoiding repetitive arithmetic computation across large daily datasets.
`waste_wastepickupimage.detections`	Pure 1NF/3NF requires a separate child table for bounding boxes.	PostgreSQL native `JSONB` column.	Bounding boxes are read as a complete unit for canvas rendering; avoids millions of relational child rows while supporting GIN indexing.
`waste_rewardsubmission.bottle_count`	Persisted derived column would violate 3NF.	Implemented as a model `@property`.	Avoids storing redundant derived data in PostgreSQL, maintaining strict 3NF compliance.

10. Normalization Summary Matrix

#	Table Name	1NF	2NF	3NF	Verified Level	Key Strategy	Integrity Safeguards
1	`auth_user`	Pass	Pass	Pass	**3NF / BCNF**	Single PK (`id`)	Unique username constraint
2	`accounts_ward`	Pass	Pass	Pass	**3NF / BCNF**	Single PK (`ward_id`)	Unique ward number constraint
3	`accounts_userprofile`	Pass	Pass	Pass	**3NF / BCNF**	Single PK (`id`)	1-to-1 unique FK to `auth_user`
4	`accounts_staff`	Pass	Pass	Pass	**3NF / BCNF**	Single PK (`staff_id`)	1-to-1 unique FK to `auth_user`
5	`accounts_recycler`	Pass	Pass	Pass	**3NF / BCNF**	Single PK (`recycler_id`)	1-to-1 unique FK to `auth_user`
6	`waste_wastepickuprequest`	Pass	Pass	Pass	**3NF / BCNF**	Single PK (`id`)	Status constraint & FK cascade
7	`waste_wastepickupimage`	Pass	Pass	Pass	**3NF / BCNF**	Single PK (`id`)	PostgreSQL JSONB binary storage
8	`waste_wastecollection`	Pass	Pass	Pass	**3NF / BCNF**	Single PK (`id`)	1-to-1 unique pickup request FK
9	`waste_wastecomplaint`	Pass	Pass	Pass	**3NF / BCNF**	Single PK (`id`)	Priority & status choices
10	`waste_staffmcfcollection`	Pass	Pass	Pass*	**2NF (Controlled 3NF)**	Single PK (`id`)	Deterministic ORM `save()` hook
11	`waste_mcfrecyclerassignment`	Pass	Pass	Pass	**3NF / BCNF**	Single PK (`id`)	Recycler vendor FK cascade
12	`waste_payment`	Pass	Pass	Pass	**3NF / BCNF**	Single PK (`id`)	Unique payment gateway token
13	`waste_rewardsubmission`	Pass	Pass	Pass	**3NF / BCNF**	Single PK (`id`)	Dynamic Python `@property`
14	`notifications_notification`	Pass	Pass	Pass	**3NF / BCNF**	Single PK (`id`)	User cascade & optional FKs

#	Table Name	1NF	2NF	3NF	Verified Level	Key Strategy	Integrity Safeguards
15	`notifications_pushsubscription`	Pass	Pass	Pass	**3NF / BCNF**	Single PK (`id`)	Unique push endpoint constraint

---

Report generated for MCA final year project documentation.